



Gestión de Bases de Datos

Clase 9: SQL - Consultas Intermedias y Avanzadas

Susana Medina Gordillo
susana.medina@correounivalle.edu.co

Continuación lenguaje SQL

Estudiantes

Atributos

Tuplas

codEst	nombre	apellido	edad	ciudad
0011	Juan	Dominguez	17	Cali
0012	Mario	Rodriguez	21	Yumbo
0013	David	Santos	20	Cali
0014	Ximena	Caicedo	18	Cali
0015	Maria	Perez	16	Cali
0016	Felipe	Holguín	23	Yumbo
0017	Juliana	Fernandez	17	Palmira
0018	Andrés	Perez	19	Jamundí

```
graph TD; Atributos --> codEst; Atributos --> nombre; Atributos --> apellido; Atributos --> edad; Atributos --> ciudad; Tuplas --> 0011; Tuplas --> 0012; Tuplas --> 0013; Tuplas --> 0014; Tuplas --> 0015; Tuplas --> 0016; Tuplas --> 0017; Tuplas --> 0018;
```

Ejemplo: Base de datos de una empresa

CLIENTES (codcli, nombre, direccion, codpostal, codciu)

VENEDORES (codven, nombre, direccion, codpostal, codpciu, codjefe)

CIUDADES (codciu, nombre, coddep)

DEPARTAMENTOS (coddep, nombre)

ARTICULOS (codart, descrip, precio, stock, stock_min, dto)

FACTURAS (codfac, fecha, codcli, codven, iva, dto)

LINEAS_FAC (codfac, linea, cant, codart, precio, dto)

Restricciones de Integridad



CLIENTES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codciu → **CIUDADES**: No acepta nulos.

VENDEDORES codjefe → **VENDEDORES** : Acepta nulos.

CIUDADES coddep → **DEPARTAMENTOS**: Acepta nulos.

FACTURAS codcli → **CLIENTES** : Acepta nulos.

FACTURAS codven → **VENDEDORES** : Acepta nulos.

LINEAS_FAC codfac → **FACTURAS** : No acepta nulos.

LINEAS_FAC codart → **ARTICULOS** : No acepta nulos.

Consultas Avanzadas

**Conectar datos de varias
tablas es esencial para
responder muchas consultas
de la vida real.**

¿Por qué SQL Avanzado?



Un simple SELECT o WHERE solo ve una tabla.

El **análisis real** necesita conectar múltiples fuentes de información.

Problema: Necesitamos saber el **nombre de los clientes** que han comprado artículos de la **descripción 'Monitor LED'** y quién fue el **vendedor** asociado.

Solución: JOINS (Uniones) y Subconsultas.

JOIN

—

Consultas a varias tablas

1. Se crea el **Producto Cartesiano** con las tablas mencionadas en la cláusula **FROM**.
2. Se aplica la cláusula **WHERE** a cada tupla del resultado del **producto cartesiano**. Las tuplas que no satisfacen la condición se **eliminan**.
3. Se aplica la cláusula **SELECT** a las tuplas resultantes.

Consultas a varias tablas:JOIN

```
codciu=2  
es Cali
```

Ej: Clientes extranjeros de la empresa.

Esto se logra a través de la cláusula **JOIN** (concatenación).

Consulta: Cuáles son los **vendedores de Cali?**

```
SELECT *  
FROM Vendedores, Ciudades  
WHERE Vendedores.codpciu=Ciudades.codciu  
      AND codciu=2  
;
```

JOIN: Ejemplos

Consulta: Cuáles son las facturas del vendedor con código "123"?

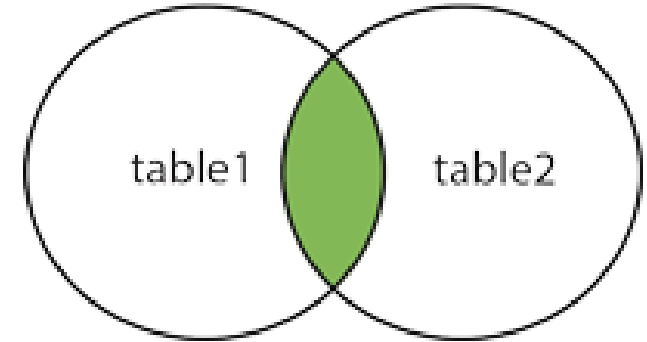
```
SELECT *  
FROM Vendedores, Facturas  
WHERE Vendedores.codven=Facturas.codven  
      AND codven="123"  
;
```

JOIN: Ejemplos

Consulta: Cuáles artículos se vendieron el 24 de diciembre de 2018?

```
SELECT *  
FROM Facturas, Lineas_Fac, Articulos  
WHERE Facturas.codfac=Lineas_Fac.codfac  
      AND Articulos.codart=Lineas_Fac.codart  
      AND fecha="24/12/2018"  
;
```

INNER JOIN

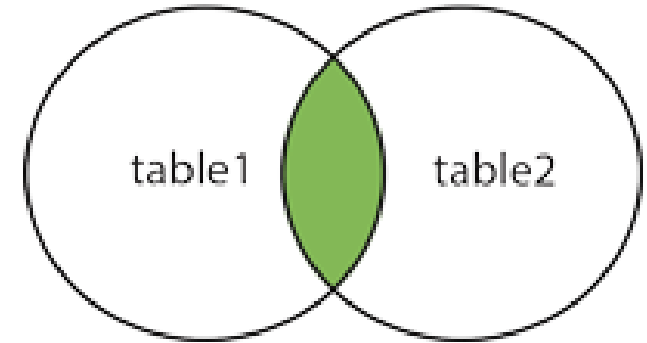


El INNER JOIN es equivalente al operador **INTERSECT**.

Devuelve solo las filas que tienen coincidencias en **ambas** tablas.

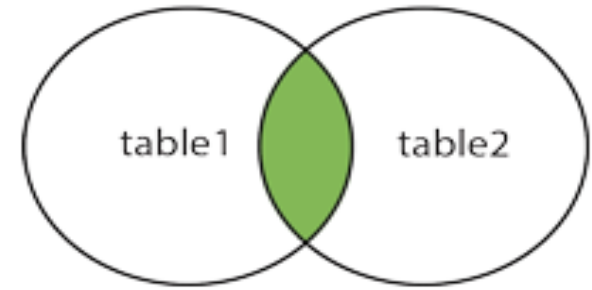
Muestra los eventos donde **todos los datos existen** (ej., solo las facturas que tienen líneas de detalle).

INNER JOIN



El INNER JOIN es equivalente al operador **INTERSECT**.

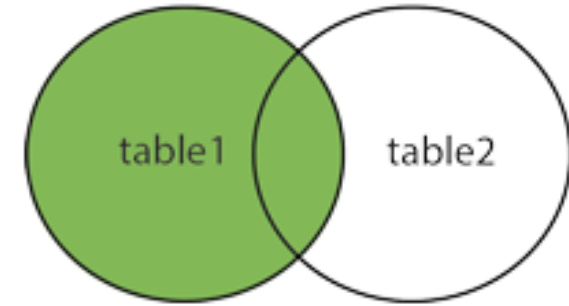
```
SELECT nombre, direccion, codciu  
  FROM Clientes  
  INNER JOIN Vendedores  
 ON Clientes.nombre = Vendedores.nombre;
```



Mostrar la fecha de cada factura junto con el nombre del vendedor que la emitió.

```
SELECT F.codfac, F.fecha, V.nombre AS Vendedor
FROM FACTURAS F
INNER JOIN VENDEDORES V ON F.codven = V.codven;
```

LEFT JOIN



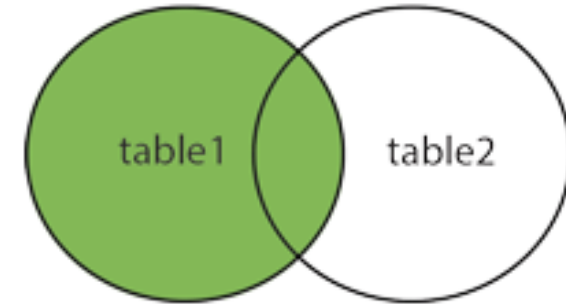
Este JOIN devuelve todas las tuplas de la tabla 1 y las tuplas de la tabla 2 que están en la **intersección**.

Devuelve todas las filas de la tabla izquierda, y las filas **coincidentes** de la tabla derecha.

Si no hay coincidencia, devuelve **NULL**.

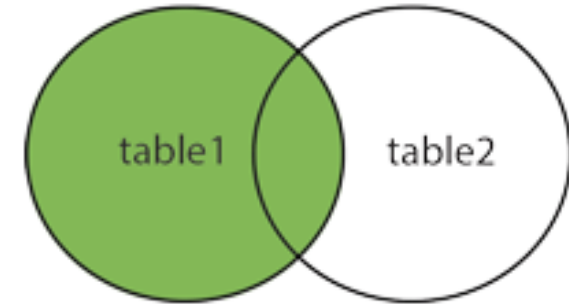
Ideal para: identificar datos faltantes o segmentos sin actividad (ej., Clientes que nunca han tenido una factura).

LEFT JOIN



```
SELECT column_name(s)
FROM table1
LEFT JOIN table2 ON table1.column_name =
table2.column_name;
```

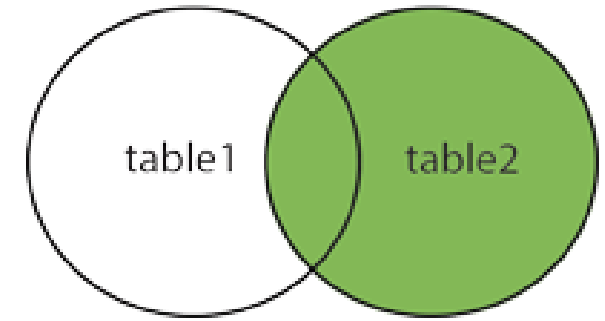
LEFT JOIN



Identificar a todos los clientes y ver cuáles aún no tienen ninguna factura asociada (*clientes sin factura*).

```
SELECT C.nombre, F.codfac
FROM CLIENTES C
LEFT JOIN FACTURAS F ON C.codcli = F.codcli
WHERE F.codfac IS NULL;
```

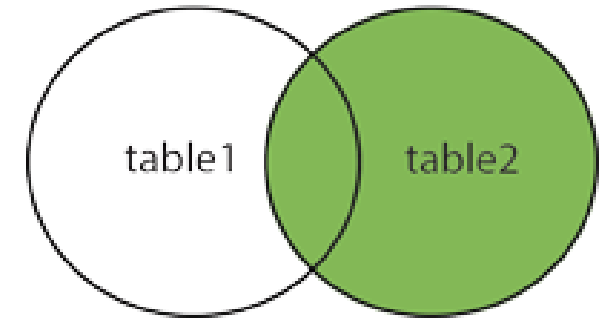
RIGHT JOIN



Este JOIN devuelve todas las tuplas de la tabla 2 y las tuplas de la tabla 1 que están en la **intersección**.

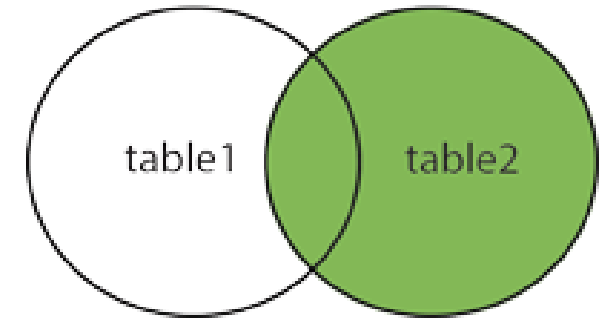
Se utiliza para asegurar que se muestren **todos los registros de la tabla derecha**, sin importar si la tabla izquierda tiene datos correspondientes.

RIGHT JOIN



```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2 ON table1.column_name =
table2.column_name;
```

RIGHT JOIN

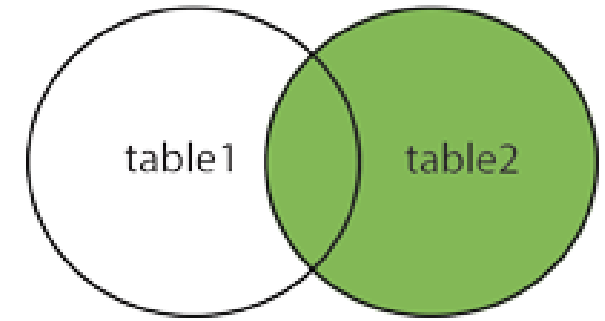


Resultado

- Todas las filas que **coinciden** entre **ambas tablas**.
- Todas las **filas únicas** de la tabla de la **derecha**.
- Valores **NULL** en las columnas de la tabla de la izquierda para aquellas filas de la derecha que no tuvieron coincidencia.

RIGHT JOIN

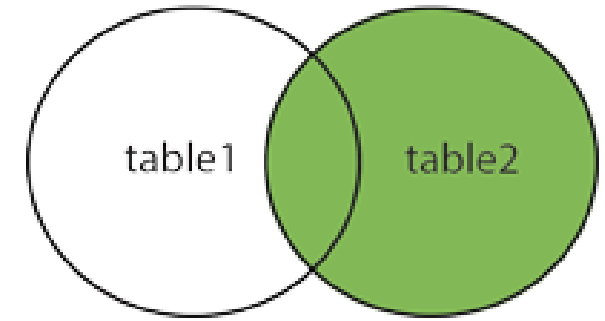
RIGHT JOIN



Esta consulta te mostraría la lista de artículos que no tienen registro de venta en la tabla LINEAS_FAC.

```
SELECT A.codart,A.descrip,LF.codfac
FROM LINEAS_FAC LF
RIGHT JOIN ARTICULOS A ON LF.codart = A.codart
WHERE LF.codfac IS NULL;
```

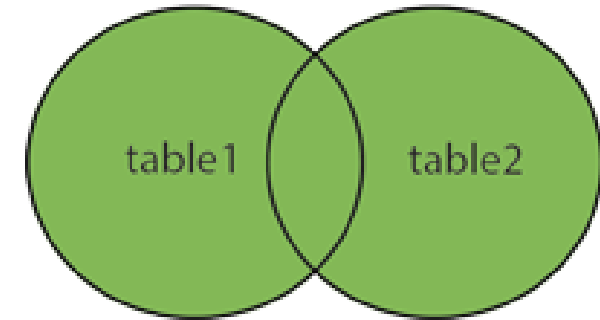
RIGHT JOIN



Este JOIN devuelve todas las tuplas de la tabla 2 y las tuplas de la tabla 1 que están en la **intersección**.

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2 ON table1.column_name =
table2.column_name;
```

FULL OUTER JOIN



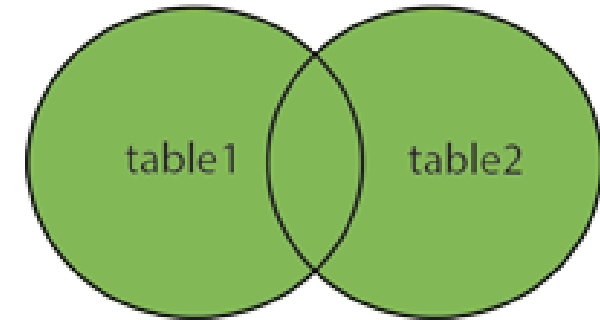
Permite retornar registros que coincidan con tuplas de la tabla 1 o de la tabla 2. Devuelve **todas las filas** cuando hay una coincidencia en **cualquiera** de las tablas.

Algunas columnas podrían quedar con valores de **NULL**.

Es la **Unión** del RIGHT y del LEFT JOIN.

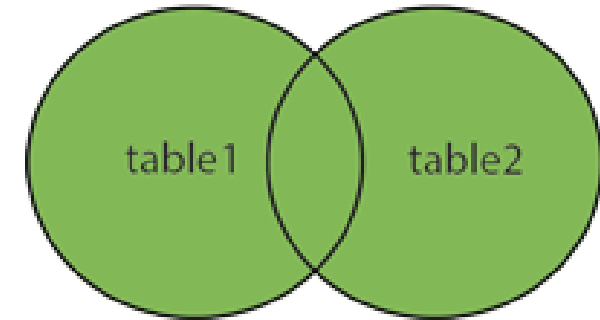
Se pueden obtener conjuntos resultado muy **grandes**!

FULL OUTER JOIN



```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2 ON table1.column_name =
table2.column_name;
```

FULL OUTER JOIN



Muestra a **TODOS** los **CLIENTES** y **TODOS** los **VENDEDORES**,
permitiendo ver fácilmente quiénes no tienen un emparejamiento.

```
SELECT C.nombre AS Cliente, V.nombre AS Vendedor
FROM CLIENTES C
FULL OUTER JOIN
    VENDEDORES V ON C.codcli = V.codjefe;
```

Self JOIN



Es un JOIN normal donde la tabla se une con sí misma.

```
SELECT column_name(s)
FROM table1 T1, table1 T2
WHERE condition;
```

Self JOIN



Listar el nombre de cada vendedor junto con el nombre de su jefe directo.

```
SELECT E.nombre AS Vendedor,  
       J.nombre AS Nombre_del_Jefe  
FROM VENDEDORES E  --empleados  
LEFT JOIN  
      VENDEDORES J  -- jefes  
ON E.codjefe = J.codven;
```

Consultas con agregación

—

Agregación

Contar, Sumar, Promediar

GROUP BY: Agrupa las filas que tienen el mismo valor en una columna para aplicar una función agregada (ej., `SUM()`, `AVG()`, `COUNT()`).

HAVING: Se usa **después** de `GROUP BY` para filtrar grupos enteros (no filas individuales, como hace `WHERE`).

Ejemplo: ventas por vendedor

```
SELECT V.nombre AS Vendedor, COUNT(F.codfac) AS  
TotalFacturas  
FROM FACTURAS F  
INNER JOIN VENDEDORES V ON F.codven = V.codven  
GROUP BY Vendedor  
HAVING COUNT(F.codfac) > 5;  
-- Solo vendedores con más de 5 facturas
```

Receso 15 min



Subconsultas

—

Subconsultas

Una subconsulta o “consulta dentro de una consulta” / consultas anidadas puede ser colocada dentro de una cláusula **WHERE** de una consulta, resultando en una expansión de las capacidades de esta cláusula.

Subconsultas: Ejemplo IN

Listar todos los artículos que han sido comprados en la línea de alguna factura.

```
SELECT descrip FROM ARTICULOS
WHERE codart IN (SELECT DISTINCT codart
                  FROM LINEAS_FAC) ;
```

Subconsultas: Ejemplo IN

Listar nombres de los vendedores de los cuales se han vendido algunos de sus productos.

```
SELECT distinct nombre
FROM VENDEDORES
WHERE codven IN
    (SELECT distinct codven
     FROM FACTURAS)
```

Subconsultas: Ejemplo

Listar nombres de los vendedores de los cuales se han vendido algunos de sus productos.

```
SELECT distinct nombre
FROM VENDEDORES
WHERE VENDEDORES.codven IN
    (SELECT distinct FACTURAS.codven
     FROM FACTURAS)
```

Subconsultas: Ejemplo



Listar los artículos cuyo precio es superior al precio promedio de **todos** los artículos.

```
SELECT descrip, precio
FROM ARTICULOS
WHERE precio > (SELECT AVG (precio) FROM
ARTICULOS) ;
```

Subconsultas correlacionadas



Esta clase de subconsultas son aquellas cuyos valores por ejecución dependen de la **fila** que está siendo examinada en la consulta principal.

La subconsulta debe generar un solo valor.

Consulta: Liste los nombres y las direcciones de los vendedores que viven en la misma ciudad que su jefe.

Es decir, que la fila del jefe a examinar depende directamente de la fila del vendedor que está siendo examinado.

Ejemplo: subconsulta correlacionada

```
SELECT A.nombre, A.direccion
FROM Vendedores A
WHERE A.codpciu =
      (SELECT B.codpciu
       FROM Vendedores B
       WHERE A.codjefe=B.codven)
;
```

codpciu es un
VARCHAR
"1"

Operadores Lógicos

—

Consultas Anidadas



IN (= ANY): Compara un valor con cualquiera de los valores devueltos por la subconsulta.

EXIST: Comprueba si la subconsulta devuelve alguna fila (**más eficiente** para verificación simple).

Ejemplo: IN – operador lógico

Los nombres de los clientes que provienen de ciudades ubicadas en los departamentos de "Valle" o "Antioquia".

```
SELECT C.nombre, CI.nombre AS Ciudad, D.nombre AS  
Departamento  
  
FROM CLIENTES C  
  
JOIN CIUDADES CI ON C.codciu = CI.codciu  
  
JOIN DEPARTAMENTOS D ON CI.coddep = D.coddep  
  
WHERE D.nombre IN ('Valle', 'Antioquia');
```

Consultas Anidadas

Otra forma de especificar subconsultas es mediante el uso del cuantificador universal **exists**.

Consulta: Identificar todas las facturas que no incluyeron el artículo con código "321".

```
SELECT  codfac  
FROM    Lineas_Fac  
WHERE   codart  != "321"
```

Consultas Anidadas

Otra forma de especificar subconsultas es mediante el uso del cuantificador universal **exists**.

Consulta: Identificar todas las facturas que no incluyeron el artículo con código “321”.

```
SELECT  codfac  
FROM    Lineas_Fac  
WHERE   codart  !=  “321”
```



Consultas Anidadas



Otra forma de especificar subconsultas es mediante el uso del cuantificador universal **EXISTS**.

Consulta: Identificar todas las facturas que no incluyeron el artículo con código "321".

```
SELECT codfac  
FROM Lineas_Fac  
WHERE codart != "321"
```

Los códigos de las facturas donde hay algunos artículos diferentes al "321".

Ejemplo: Consultas Anidadas

En el caso anterior se debe usar **NOT EXISTS**:

```
SELECT codfac
FROM Facturas
WHERE NOT EXISTS
  (SELECT codfac
   FROM Lineas_Fac
   WHERE codart = "321" AND
    Facturas.codfac = Lineas_Fac.codfac
  )
```


Consultas Anidadas: EXISTS

Los operadores **EXISTS** y **NOT EXISTS** siempre preceden la subconsulta.

El **EXISTS** evalúa como “verdadero” si el conjunto resultante de la subconsulta no es vacío, de lo contrario lo evalúa como “falso”. El operador **NOT EXISTS**, trabaja de una forma opuesta.

Porqué se debería usar exists o not exists?

La opción **EXISTS** provee el único medio disponible para resolver consultas que contengan el cuantificador “**TODOS**” en la condición. En el álgebra relacional, tales consultas se resolvían con la *división*.

Comparación de Conjuntos

Cómo comparar conjuntos de las consultas anidadas?

Ejemplo: Determinar el código de todos los artículos vendidos con un descuento mayor que, al menos, un artículo vendido el 31 de diciembre.

```
SELECT distinct T.codart
FROM Lineas_Fac AS T, Lineas_Fac AS S,
Facturas AS Q
WHERE Q.fecha = "31/12/2018" AND Q.codfac =
T.codfac
AND T.dto > S.dto;
```

Búsqueda en cadenas (texto)

—

Comparación de Cadenas



Cuando se necesita información sobre las filas que tengan valores similares en una columna determinada se usa **LIKE** en la cláusula **WHERE**.

No se requiere conocer el valor de la columna completa para identificar la fila y escogerla.

WHERE < atributo> **LIKE** modelo

SQL debe proporcionar alguna idea de donde está la cadena de caracteres parcial. Se usan dos caracteres especiales a modo de comodín, que forman parte del modelo. Los caracteres son % (porcentaje) y _ (guión bajo).

LIKE



El (%) se interpreta como un carácter que puede representar cualquier cadena de caracteres de cualquier longitud (hasta de tamaño cero).

Ejemplo: Listar las ciudades que empiecen por **Ca** (Cali, Cartago, Cartagena, ...)

```
SELECT *  
FROM Ciudades  
WHERE nombre LIKE "Ca%";
```

LIKE



El (`_`) permite comparar en los lugares que aparezca el símbolo `_`.

Ejemplo: Listar el **nombre** de los empleados cuyo código de supervisor tiene los números **1** en el primer carácter y **3** en el tercer carácter.

```
SELECT nombre
```

```
FROM Empleado
```

```
WHERE codSuperv LIKE '1_3_';
```

Comparación de Fecha y Hora

Los valores de fecha pueden representarse en varios formatos como **04/19/1970** o **19 Abril 1970**.

Cada SGBD permite definir estos formatos. Los tipos de datos más usados son **DATE** (fecha) y **TIMESTAMP** (hora y fecha).

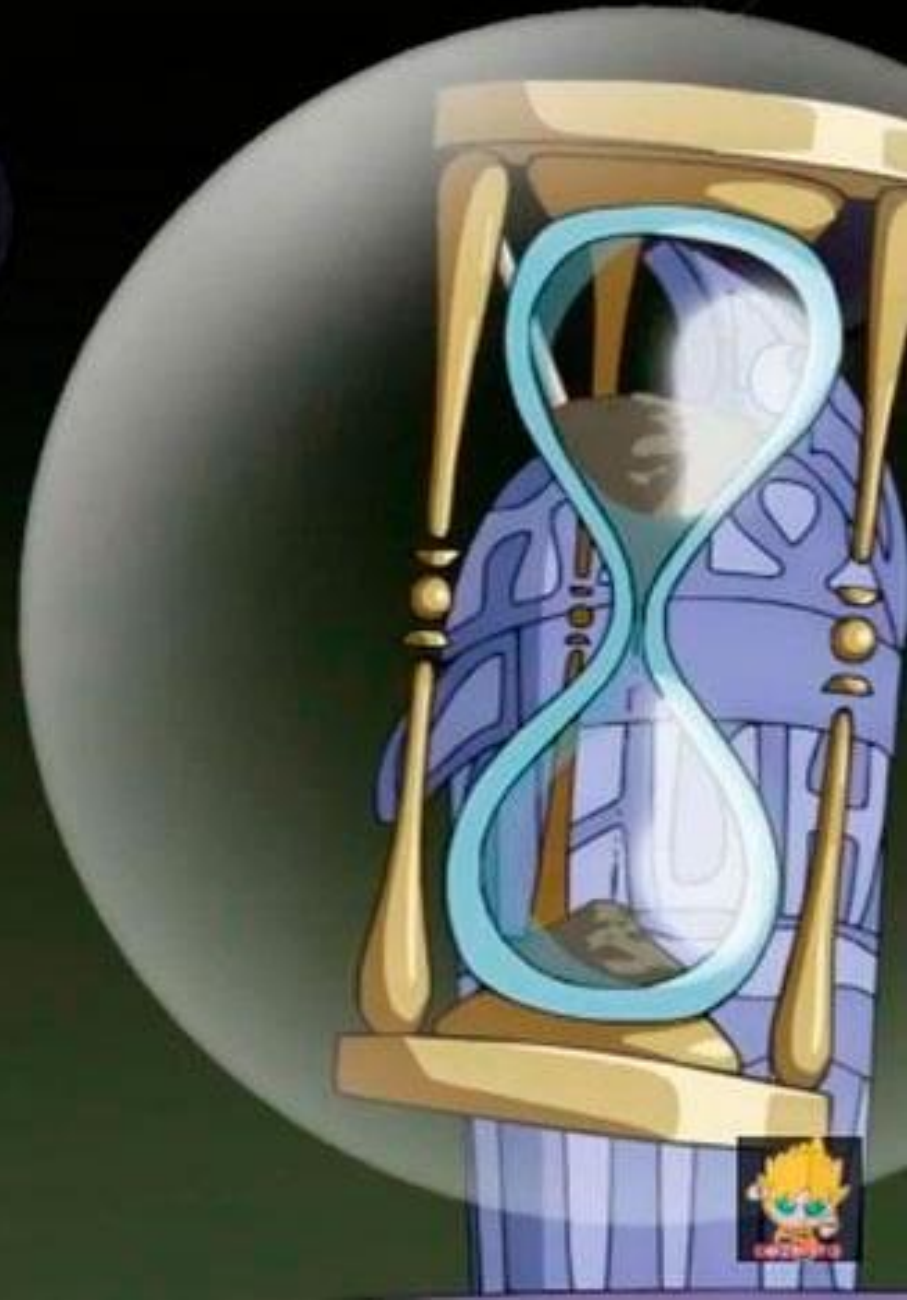
Se pueden **comparar fechas** u horas usando los operadores de comparación conocidos.

Existen las funciones: DAY, MONTH y YEAR para obtener estos datos a partir de una fecha:

```
SELECT DAY ( ' 2019/02/15 ' ) AS DayOfMonth;
```

9 : 26

Cuestionario



Ejercicio Práctico